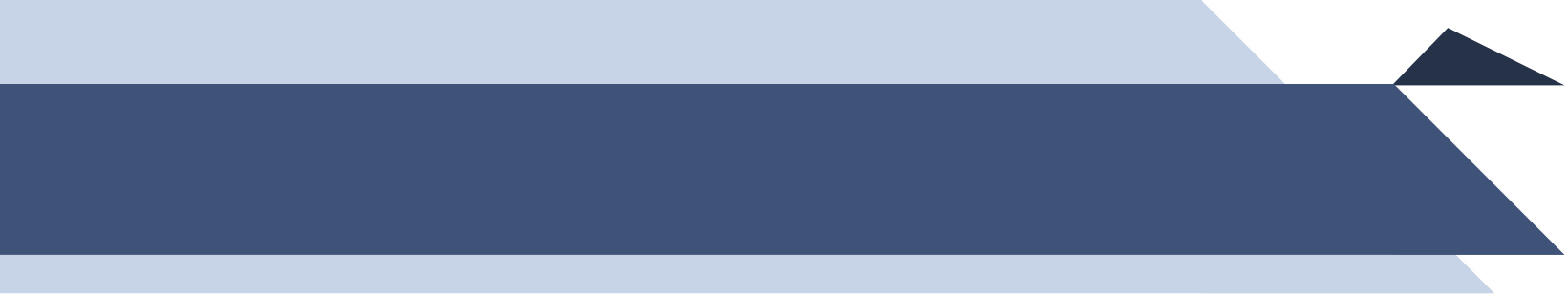


Enterprise JAVA (22MCA202)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte



Unit 3

3. Event Handling





Java Applet

- Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantages of Applet

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

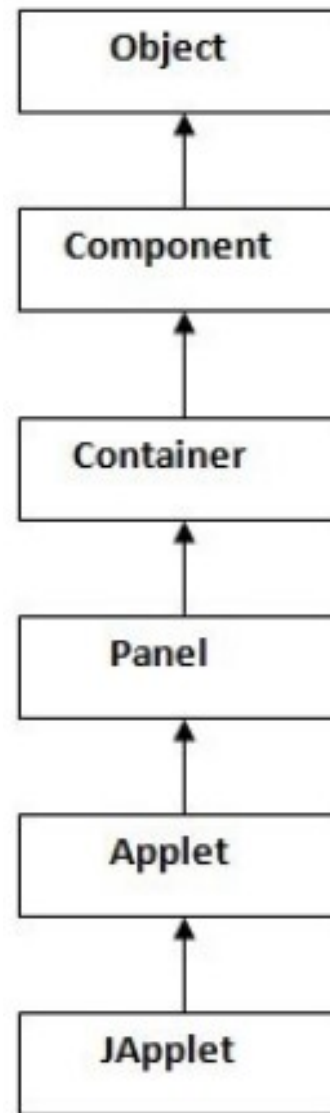
Drawback of Applet

- Plugin is required at client browser to execute applet.
- 

Differences between an Applet and a stand-alone Java application

- A main() method is not invoked on an applet, and an applet class will not define main().
- Applets are designed to be embedded within an HTML page.
- When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.
- A JVM is required to view an applet. The JVM can be either a plug-in of the Web browser or a separate runtime environment.
- The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.

Hierarchy of Applet





Applet Basics

- Applets are Small applications that are accessed on an Internet server, transported over the Internet, automatically installed and run as a part of a web document.
- After an applet arrives on the client, it has limited access to resources so that it can produce an arbitrary multimedia user interface.

Begins with two import statements

Import java.awt.*; :-- imports abstract window toolkit (AWT) classes

Applet interacts with user through AWT, not through console based I/O classes.

Import java.applet.*; :-- imports Applet class

All the applets that you create must be subclasses of Applet class.



Applet Life Cycle

Applet Lifecycle



1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

Life cycle methods for Applet

The *java.applet.Applet* class provides 4 life cycle methods and *java.awt.Component* class provides 1 life cycle method for an applet.

java.applet.Applet class

For creating any applet java.applet.Applet class must be inherited. It provides 4 life cycle methods of applet.

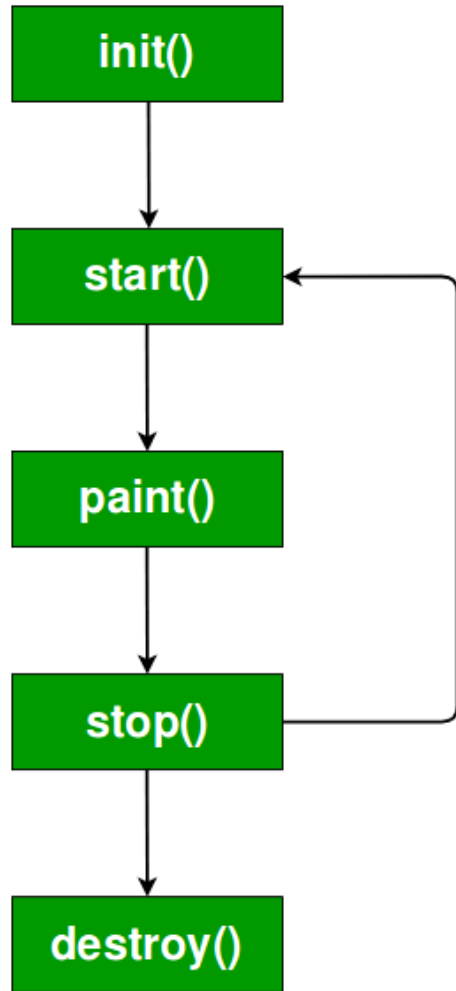
1. **public void init():** is used to initialize the Applet. It is invoked only once.
2. **public void start():** is invoked after the init() method or browser is maximized. It is used to start the Applet.
3. **public void stop():** is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
4. **public void destroy():** is used to destroy the Applet. It is invoked only once.

java.awt.Component class

The Component class provides 1 life cycle method of applet.

1. **public void paint(Graphics g):** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Life cycle methods for Applet



It is important to understand the order in which the various methods shown in the above image are called.

When an applet begins, the following methods are called, in this sequence:

1. `init()`
2. `start()`
3. `paint()`

When an applet is terminated, the following sequence of method calls takes place:

1. `stop()`
2. `destroy()`



AWT Event Handling

What is an Event?

- Change in the state of an object is known as event.
 - i.e. event describes the change in state of source.
 - Events are generated as result of user interaction with the graphical user interface components.
 - For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page are the activities that causes an event to happen.
- 



Types of Events

The events can be broadly classified into two categories:

Foreground Events -

Those events which require the direct interaction of user. They are generated as consequences of a person interacting with the graphical components in Graphical User Interface.

For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page etc.

Background Events -

Those events that require the interaction of end user are known as background events. Operating system interrupts, hardware or software failure, timer expires, an operation completion are the example of background events.





Event Handling

Event Handling is the mechanism that controls the event and decides what should happen if an event occurs.

This mechanism have the code which is known as *event handler* that is executed when an event occurs.

Java Uses the *Delegation Event Model* to handle the events.

This model defines the standard mechanism to generate and handle the events.





Delegation Event Model

The Delegation Event Model has the following key participants namely:

Source - The source is an object on which event occurs.

Source is responsible for providing information of the occurred event to its handler. Java provides as with classes for source object.

Listener - It is also known as event handler.

Listener is responsible for generating response to an event. From Java implementation point of view the listener is also an object.

Listener waits until it receives an event. Once the event is received, the listener processes the event and then returns.

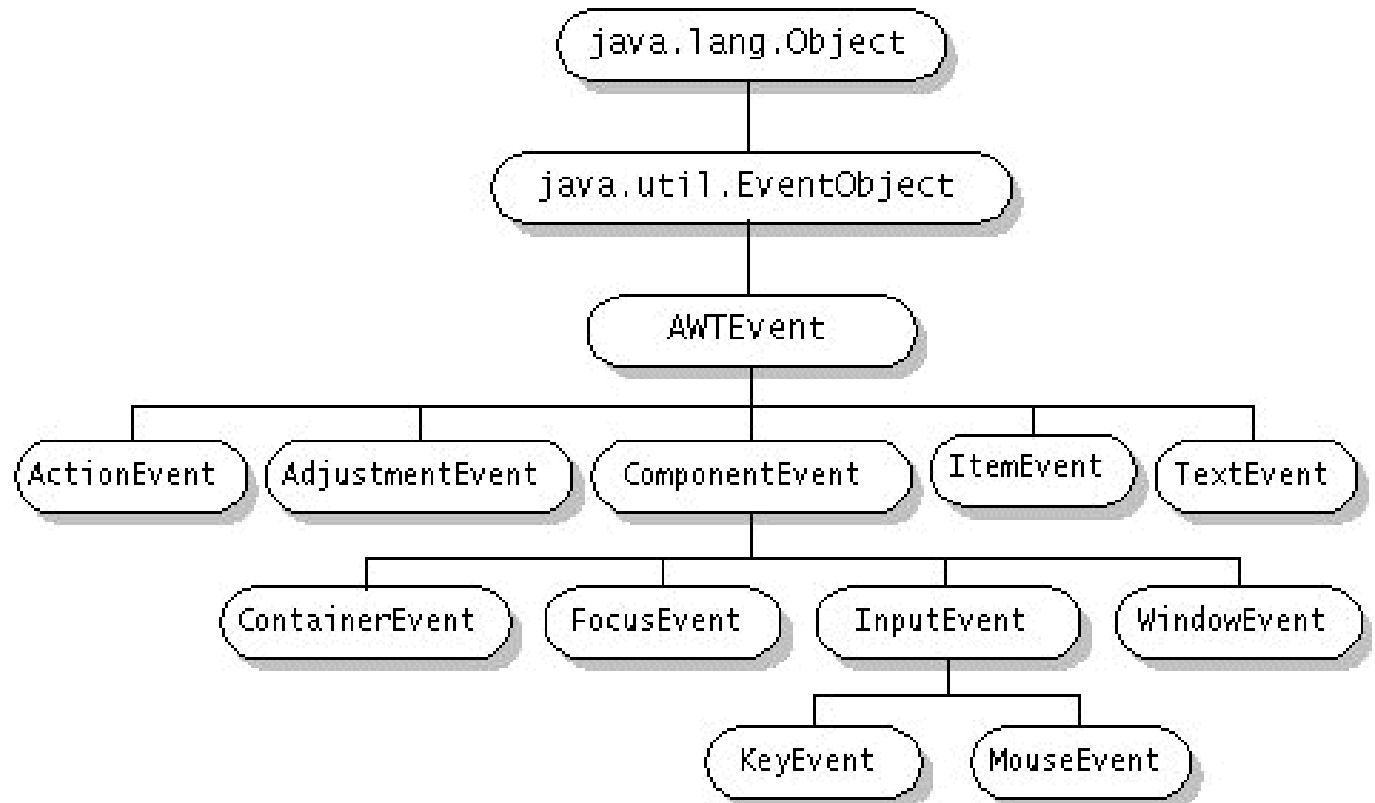
In this model, Listener needs to be registered with the source object so that the listener can receive the event notification.



Event Classes

The class **AWTEvent**, is defined within the java.awt package.

The package **java.awt.event** defines several types of events that are generated by various user interface elements.



Event Classes

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double clicked, a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when a mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Sources of Events

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Event Listener Interfaces

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Java Event classes and Listener interfaces

Event Classes	Listener Interfaces
ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

Using the Delegation Event Model

Using the delegation event model follows **two steps**:

1. Implement the appropriate interface in the listener so that it will receive the type of event desired.
2. Implement code to register and un-register(if necessary) the listener as a recipient for the event notifications.

Remember that a source may generate several types of events. Each event must be registered separately.

Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.



Swings

- Swings is used to create window-based applications.
- Java Swing provides platform-independent and lightweight components.
- The javax.swing package provides classes for java swing API such as JLabel, JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.





JApplet Class

- The JApplet class extends the Applet class.
- JApplet is the public class of swing.
- JApplet generates a bytecode with the help of JVM or the Applet viewer.
- JApplet supports various “panes” such as **content pane, glass pane and root pane**.
- When adding component to an instance of JApplet, the **add()** method is called for the content pane of the JApplet object.
- The content pane can be obtained using the method, ***Container getContentPane()***
- The **add()** method of container is then used to add a component to a content pane.
- **General Format:**

void add(comp)

where comp is the component to be added to the content pane.





Java JFrame

- The Java JFrame is an essential component of Java Swing, which is a part of the Java SWT(Standard Widget Toolkit).
- JFrame in Java is a class that allows you to create and manage a top-level window in a Java application.
- It serves as the main window for GUI-based Java applications and provides a platform-independent way to create graphical user interfaces.
- In Java JFrame is a part of javax.swing package.



Java JFrame

Methods of Java JFrame

Methods	Description
<code>setTitle(String title)</code>	Sets the title of the JFrame.
<code>setSize(int width, int height)</code>	Sets the size of the JFrame.
<code>setDefaultCloseOperation(int operation)</code>	Sets the default close operation for the JFrame. Common options include <code>JFrame.EXIT_ON_CLOSE</code> , <code>JFrame.HIDE_ON_CLOSE</code> , and <code>JFrame.DO_NOTHING_ON_CLOSE</code> .
<code>setVisible(boolean b)</code>	Sets the visibility of the JFrame. Pass true to make it visible and false to hide it.
<code>setLayout(LayoutManager manager)</code>	Sets the layout manager for the JFrame, which controls how components are arranged within the frame.



Java JFrame

add(Component comp)	Adds a Swing component to the JFrame.
remove(Component comp)	Removes a component from the JFrame.
validate()	Forces the layout manager to recalculate the layout of components within the JFrame.
setResizable(boolean resizable)	Controls whether the user can resize the JFrame.
setIconImage(Image image)	Sets the icon (image) for the JFrame window.



JApplets Vs JFrame

Feature	JApplet	JFrame
Purpose	Used for creating Java applets	Used for creating standalone GUI applications
Deployment	Embedded within web browsers	Runs independently of web browsers
Lifecycle	Managed by the browser or applet viewer	Controlled by programmer's code
HTML Embedding	Embedded within HTML using <applet> tag	Not embedded within HTML
Window Decorations	Typically no window decorations	Includes title bar, borders, and control buttons
Usage	Declined due to security concerns and alternatives	Commonly used for modern GUI applications in Java

Java JLabel

- The object of JLabel class is a component for placing text in a container.
- It is used to display a single line of read only text.
- The text can be changed by an application but a user cannot edit it directly.
- It inherits JComponent class.

Commonly used Constructors:

Constructor	Description
<code>JLabel()</code>	Creates a JLabel instance with no image and with an empty string for the title.
<code>JLabel(String s)</code>	Creates a JLabel instance with the specified text.
<code>JLabel(Icon i)</code>	Creates a JLabel instance with the specified image.
<code>JLabel(String s, Icon i, int horizontalAlignment)</code>	Creates a JLabel instance with the specified text, image, and horizontal alignment.

Java JLabel

Commonly used Methods:

Methods	Description
String getText()	It returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment(int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.

Java JButton

- The JButton class is used to create a labeled button that has platform independent implementation.
- The application result in some action when the button is pushed.
- It inherits AbstractButton class.

Commonly used Constructors:

Constructor	Description
JButton()	It creates a button with no text and icon.
JButton(String s)	It creates a button with the specified text.
JButton(Icon i)	It creates a button with the specified icon object.

Java JButton

Commonly used Methods of AbstractButton class:

Methods	Description
<code>void setText(String s)</code>	It is used to set specified text on button
<code>String getText()</code>	It is used to return the text of the button.
<code>void setEnabled(boolean b)</code>	It is used to enable or disable the button.
<code>void setIcon(Icon b)</code>	It is used to set the specified Icon on the button.
<code>Icon getIcon()</code>	It is used to get the Icon of the button.
<code>void setMnemonic(int a)</code>	It is used to set the mnemonic on the button.
<code>void addActionListener(ActionListener a)</code>	It is used to add the action listener to this object.

Java JTextField

- The object of a JTextField class is a text component that allows the editing of a single line text.
- It inherits JTextComponent class.

Commonly used Constructors:

Constructor	Description
JTextField()	Creates a new TextField
JTextField(String text)	Creates a new TextField initialized with the specified text.
JTextField(String text, int columns)	Creates a new TextField initialized with the specified text and columns.
JTextField(int columns)	Creates a new empty TextField with the specified number of columns.

Java JTextField

Commonly used Methods:

Methods	Description
<code>void addActionListener(ActionListener l)</code>	It is used to add the specified action listener to receive action events from this textfield.
<code>Action getAction()</code>	It returns the currently set Action for this ActionEvent source, or null if no Action is set.
<code>void setFont(Font f)</code>	It is used to set the current font.
<code>void removeActionListener(ActionListener l)</code>	It is used to remove the specified action listener so that it no longer receives action events from this textfield.

Java JCheckBox

- The JCheckBox class is used to create a checkbox.
- It is used to turn an option on (true) or off (false).
- Clicking on a CheckBox changes its state from "on" to "off" or from "off" to "on".
- It inherits JToggleButton class.

Commonly used Constructors:

Constructor	Description
JCheckBox()	Creates an initially unselected check box button with no text, no icon.
JChechBox(String s)	Creates an initially unselected check box with text.
JCheckBox(String text, boolean selected)	Creates a check box with text and specifies whether or not it is initially selected.
JCheckBox(Action a)	Creates a check box where properties are taken from the Action supplied.

Java JCheckBox

Commonly used Methods:

Methods	Description
<code>AccessibleContext getAccessibleContext()</code>	It is used to get the <code>AccessibleContext</code> associated with this <code>JCheckBox</code> .
<code>protected String paramString()</code>	It returns a <code>string</code> representation of this <code>JCheckBox</code> .

Java JRadioButton

- The JRadioButton class is used to create a radio button.
- It is used to choose one option from multiple options.
- It should be added in ButtonGroup to select one radio button only.

Commonly used Constructors:

Constructor	Description
JRadioButton()	Creates an unselected radio button with no text.
JRadioButton(String s)	Creates an unselected radio button with specified text.
JRadioButton(String s, boolean selected)	Creates a radio button with the specified text and selected status.

Java JRadioButton

Commonly used Methods:

Methods	Description
<code>void setText(String s)</code>	It is used to set specified text on button.
<code>String getText()</code>	It is used to return the text of the button.
<code>void setEnabled(boolean b)</code>	It is used to enable or disable the button.
<code>void setIcon(Icon b)</code>	It is used to set the specified Icon on the button.
<code>Icon getIcon()</code>	It is used to get the Icon of the button.
<code>void setMnemonic(int a)</code>	It is used to set the mnemonic on the button.
<code>void addActionListener(ActionListener a)</code>	It is used to add the action listener to this object.

Examples: `GUIApplet.java`, `GUIFrame.java`

The KeyEvent Class

A KeyEvent is generated when keyboard input occurs.

There are three types of key events, which are identified by these integer constants: **KEY_PRESSED**, **KEY_RELEASED**, and **KEY_TYPED**.

The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated.

Remember, not all key presses result in characters. For example, pressing the SHIFT key does not generate a character.

There are many other integer constants that are defined by KeyEvent. For example, **VK_0 through VK_9** and **VK_A through VK_Z** define the ASCII equivalents of the numbers and letters. Here are some others:

VK_ENTER

VK_ESCAPE

VK_CANCEL

VK_UP

VK_DOWN

VK_LEFT

VK_RIGHT

VK_PAGE_DOWN

VK_PAGE_UP

VK_SHIFT

VK_ALT

VK_CONTROL

The KeyEvent Class

Constructors:

`KeyEvent(Component src, int type, long when, int modifiers, int code)`

`KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)`

Here, **src** is a reference to the component that generated the event.

The type of the event is specified by **type**.

The system time at which the key was pressed is passed in **when**.

The **modifiers** argument indicates which modifiers were pressed when this key event occurred.

The virtual key code, such as `VK_UP`, `VK_A`, and so forth, is passed in **code**.

The character equivalent (if one exists) is passed in **ch**. If no valid character exists, then `ch` contains `CHAR_UNDEFINED`. For `KEY_TYPED` events, `code` will contain `VK_UNDEFINED`.



The KeyEvent Class

Methods:

The KeyEvent class defines several methods, but the most commonly used ones are `getKeyChar()` and `getKeyCode()`,

`char getKeyChar()` - returns the character that was entered

`int getKeyCode()` - which returns the key code

If no valid character is available, then `getKeyChar()` returns `CHAR_UNDEFINED`.

When a `KEY_TYPED` event occurs, `getKeyCode()` returns `VK_UNDEFINED`.



The KeyListener Interface

S.N.	Method & Description
1	void keyPressed(KeyEvent e) Invoked when a key has been pressed.
2	void keyReleased(KeyEvent e) Invoked when a key has been released.
3	void keyTyped(KeyEvent e) Invoked when a key has been typed.

For example, if a user presses and releases the a key, three events are generated in sequence: key pressed, typed, and released. If a user presses and releases the HOME key, two key events are generated in sequence: key pressed and released.

Example: [KeyboardEventApplet.java](#)



Adapter Classes

- Java provides a special feature, called an adapter class, that can simplify the creation of event handlers in certain situations.
 - An adapter class provides an empty implementation of all methods in an event listener interface
 - i.e this class itself write definition for methods which are present in particular event listener interface.
 - However these definitions does not affect program flow or meaning at all.
 - Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface.
 - You can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which you are interested.
- 

Adapter Classes

java.awt.event Adapter classes

Adapter class	Listener interface
WindowAdapter	WindowListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
FocusAdapter	FocusListener
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
HierarchyBoundsAdapter	HierarchyBoundsListener

Example: AdapterKeyEvents.java